CM2005 Report

Task 1

Under task1.cpp file

```cpp
// Define a Candlestick class
class Candlestick {
public:
    double open; // The average temperature of the previous year
    double close; // The average temperature of the current year
    double high; // The highest temperature of the current year
    double low; // The lowest temperature of the current year
    int year;

    // Constructor
    Candlestick(int yr, double op, double cl, double hi, double lo)
        : year(yr), open(op), close(cl), high(hi), low(lo) {}
};
```

This class is the blueprint for storing candlestick data. A constructor initializes these values when an object is created. Instances of Candlestick are stored in a std:: vector for easy access and iteration.

```cpp
// Function to safely convert a string to double
double safeStringToDouble(const std::string& str) {
    try {
        if (str.empty()) throw std::invalid_argument("Empty string");
        return std::stod(str);
    } catch (const std::exception&) {
        return std::numeric_limits<double>::quiet_NaN();
    }
}
```

This part converts a string to a double while handling errors gracefully to ensure robust handling of invalid or empty string inputs during temperature parsing. It uses std:: stod to convert the string to a double. It returns a special value (NaN) for invalid data, which is ignored during processing.

Compute Candlestick Data

Part 1: Open the file

```cpp
// Function to compute candlestick data
std::vector<Candlestick> computeCandlestickData(const std::string& location, const std::string& filePath) {
    std::vector<Candlestick> candlesticks;
    std::ifstream file(filePath);

    if (!file.is_open()) {
        std::cerr << "Error: Unable to open file: " << filePath << std::endl;
        return candlesticks;
    }
```

This part opens the dataset file and initializes an empty vector to store the candlestick data. If the file cannot be opened, an error message is displayed, and the function returns an empty vector.

Part 2: Parsing the Header Line

```
std::string line;
std::map<std::string, int> locationColumnIndex;
```

```
// Read the header line and map column indices
if (std::getline(file, line)) {
    std::stringstream ss(line);
    std::string columnName;
    int index = 0;
    while (std::getline(ss, columnName, ',')) {
        locationColumnIndex[columnName] = index++;
    }
}
```

This part reads the first line of the CSV file to map column names to their respective indices. It creates a mapping (locationColumnIndex) from column names (e.g., GB_temperature) to their index positions. This allows quick identification of the column for the specified location.

Part 3: Verify Location Exists

```
// Ensure the specified location exists in the dataset
if (locationColumnIndex.find(location) == locationColumnIndex.end()) {
    std::cerr << "Error: Location '" << location << "' not found in the dataset.\n";
    return candlesticks;
}

int columnIndex = locationColumnIndex[location];
```

This ensures the specified location exists in the dataset. If the location is not found, an error message is displayed, and the function exits early. It also retrieves the index of the target column for quick access.

Part 4: Process Each Line of Data

```
std::vector<double> previousYearTemps;
std::vector<double> currentYearTemps;
int currentYear = 0;
double yearlyHigh = std::numeric_limits<double>::lowest();
double yearlyLow = std::numeric_limits<double>::max();
```

This part initializes variables for tracking temperature data across years.
previousYearTemps: Stores temperatures from the previous year (used for calculating open)

currentYearTemps: Stores temperatures for the current year.

yearlyHigh and yearlyLow: Track the highest and lowest temperatures for the current year.

Part 5: Parse and Extract Data

```
// Process each line of data
while (std::getline(file, line)) {
    std::stringstream ss(line);
    std::string date, temperatureStr;
    double temperature;
    int columnCounter = 0;

    // Parse date
    std::getline(ss, date, ',');

    // Locate the correct temperature column
    for (int i = 0; i <= columnIndex; ++i) {
        std::getline(ss, temperatureStr, ',');
    }

    temperature = safeStringToDouble(temperatureStr);
    if (std::isnan(temperature)) continue;

    // Extract the year from the date (format assumed: YYYY-MM-DD)
    int year = std::stoi(date.substr(0, 4));
```

This part extracts and processes data for each line as mentioned in comment above. If previous year temperature value is not detected, an output of “nan” will be displayed. (e.g: Year 1980)

Part 6: Handle Year Changes and update Current Year Data

```
// Handle year change
if (currentYear != year) {
    if (currentYear != 0 && !currentYearTemps.empty()) {
        // Compute 'open' as the average temperature of the previous year
        double open = !previousYearTemps.empty() ?
                      std::accumulate(previousYearTemps.begin(), previousYearTemps.end(), 0.0) / previousYearTemps.size() :
                      std::numeric_limits<double>::quiet_NaN();
        // Compute 'close' as the average temperature of the current year
        double close = std::accumulate(currentYearTemps.begin(), currentYearTemps.end(), 0.0) / currentYearTemps.size();

        // Add a new candlestick object for the completed year
        candlesticks.emplace_back(currentYear, open, close, yearlyHigh, yearlyLow);
    }

    // Transition to the new year: move current year's temperatures to the previous year
    previousYearTemps = currentYearTemps;
    currentYearTemps.clear();

    // Reset yearly high and low temperatures
    currentYear = year;
    yearlyHigh = std::numeric_limits<double>::lowest();
    yearlyLow = std::numeric_limits<double>::max();
}
```

This part detects year transitions and computes candlestick data for the completed year, adds the current temperature to currentYearTemps and updates yearlyHigh and yearlyLow based on the new temperature.

Part 7: Handle Last Year’s Data

```
// Handle the last year in the dataset
if (!currentYearTemps.empty()) {
    double open = !previousYearTemps.empty() ?
                  std::accumulate(previousYearTemps.begin(), previousYearTemps.end(), 0.0) / previousYearTemps.size() :
                  std::numeric_limits<double>::quiet_NaN();
    double close = std::accumulate(currentYearTemps.begin(), currentYearTemps.end(), 0.0) / currentYearTemps.size();

    candlesticks.emplace_back(currentYear, open, close, yearlyHigh, yearlyLow);
}
```

This part ensures the last year's data is processed after exiting the loop.

Part 8: Return Result

```
  return candlesticks;
```

This part returns the computed vector of Candlestick objects for further use or display.

```
// Helper function to print candlestick data
void printCandlestickData(const std::vector<Candlestick>& candlesticks) {
    std::cout << std::setw(8) << "Year" << std::setw(12) << "Open"
              << std::setw(12) << "Close" << std::setw(12) << "High"
              << std::setw(12) << "Low" << std::endl;
    std::cout << std::string(56, '-') << std::endl;

    for (const auto& c : candlesticks) {
        std::cout << std::setw(8) << c.year
                  << std::setw(12) << std::fixed << std::setprecision(2) << c.open
                  << std::setw(12) << c.close
                  << std::setw(12) << c.high
                  << std::setw(12) << c.low << std::endl;
    }
}
```

This part displays the column headers (Year, Open, Close, High, Low) aligned with the respective data columns, prints a separator line (- repeated 56 times) for better visual distinction between the header and the data, std::setw(8) and std::setw(12) ensure consistent column widths (8 for the year, 12 for the others), making the table visually uniform. Lastly, it loops through the candlesticks vector to print each candlestick's data (year, open, close, high, low).

Main function:

```
// Main function
int main() {
    std::string filePath = "weather_data_EU_1980-2019_temp_only.csv";

    // Get the country code from the user
    std::string location;
    std::cout << "Enter countrycode_temperature (e.g., GB_temperature): ";
    std::cin >> location;

    std::vector<Candlestick> candlestickData = computeCandlestickData(location, filePath);

    if (candlestickData.empty()) {
        std::cout << "No data available for the specified country." << std::endl;
        return 1;
    }

    printCandlestickData(candlestickData);

    return 0;
}
```

This part is the entry point of the program to integrate user interaction, file reading, data processing, and result display. It specifies the dataset file path, prompts the user for a location and calls computeCandlestickData with the user-provided location and file path. If no data is available, displays an error message. Otherwise, callsprintCandlestickData to display the results.

Command:

```
PS C:\Y2S1\object-oriented program\oop> g++ task1.cpp -o main
PS C:\Y2S1\object-oriented program\oop> ./main
```

Prompt:

```
Enter countrycode_temperature (e.g., GB_temperature): CH_temperature
```

Output:

Year	Open	Close	High	Low
1980	nan	5.84	27.79	-17.00
1981	5.84	6.99	27.58	-19.07
1982	6.99	7.63	27.80	-17.37
1983	7.63	8.23	34.56	-18.41
1984	8.23	6.98	33.54	-13.24
1985	6.98	6.27	29.22	-21.46
1986	6.27	7.09	31.70	-18.60
1987	7.09	6.13	27.45	-25.42
1988	6.13	7.77	31.89	-11.45
1989	7.77	8.38	32.42	-12.78
1990	8.38	8.28	32.03	-12.78
1991	8.28	7.04	32.33	-16.90
1992	7.04	8.09	35.04	-14.67
1993	8.09	7.49	31.98	-19.91
1994	7.49	8.75	35.53	-15.41
1995	8.75	7.47	32.73	-16.22
1996	7.47	5.64	28.45	-23.79
1997	5.64	7.00	30.36	-17.58
1998	7.00	7.88	33.69	-15.82
1999	7.88	8.01	30.55	-14.04
2000	8.01	8.88	33.58	-15.21
2001	8.88	7.48	30.44	-17.45
2002	7.48	8.32	31.86	-20.88
2003	8.32	8.19	36.20	-17.33
2004	8.19	7.71	31.54	-16.63
2005	7.71	7.40	33.85	-16.71
2006	7.40	7.88	32.81	-26.14
2007	7.88	9.05	34.53	-9.83
2008	9.05	8.77	29.85	-10.82
2009	8.77	8.21	30.84	-14.76
2010	8.21	6.55	31.85	-21.71
2011	6.55	8.39	32.13	-13.99
2012	8.39	8.25	35.72	-21.71
2013	8.25	7.70	33.97	-15.39
2014	7.70	9.55	32.28	-12.85
2015	9.55	9.45	36.22	-9.95
2016	9.45	8.68	31.40	-11.96
2017	8.68	8.46	34.13	-16.32
2018	8.46	9.80	33.13	-14.78
2019	9.80	9.66	34.24	-11.41

Task 2

Under task2.cpp file

It computes Candlestick Data similarly to task 1.

```cpp
// Function to print vertical candlestick plot with values
void printVerticalCandlestickPlot(const std::vector<Candlestick>& candlesticks) {
    const int plotHeight = 10;

    for (const auto& c : candlesticks) {
        // Print the legend for this year's data
        std::cout << "Legend:\n";
        std::cout << "  H - High temperature\n";
        std::cout << "  O - Open (average temperature of the previous year)\n";
        std::cout << "  C - Close (average temperature of the current year)\n";
        std::cout << "  L - Low temperature\n";
        std::cout << std::string(30, '-') << "\n";

        std::cout << "Year " << c.year << ":\n";
        std::cout << "  Open: " << c.open << " | Close: " << c.close
                  << " | High: " << c.high << " | Low: " << c.low << "\n";
```

This part displays the candlestick data as a vertical plot using ASCII characters and includes the legend for the characters.

1. Temperature Range Mapping:

```cpp
    int openPos = static_cast<int>((c.open - c.low) / range * (plotHeight - 1));
    int closePos = static_cast<int>((c.close - c.low) / range * (plotHeight - 1));
```

This maps high, low, open, and close temperature values to specific rows in the plot.

2. Generate Scale and Rows:

```cpp
  // Adjust min and max temperatures to the nearest multiples of 10
  int minTemp = static_cast<int>(std::floor(c.low / 10.0)) * 10;
  int maxTemp = static_cast<int>(std::ceil(c.high / 10.0)) * 10;
```

This part adjusts temperatures (y-axis) to a scale of 10 for better readability.

3. Print Vertical Plot:

```cpp
for (int i = 0; i < plotHeight; ++i) {
    // Print temperature value if it matches the row
    if (tempRows.find(i) != tempRows.end()) {
        std::cout << std::setw(4) << tempRows[i] << " ";
    } else {
        std::cout << "     "; // Empty space for alignment
    }

    // Display candlestick representation
    if (i == openPos) std::cout << " O  ";
    else if (i == closePos) std::cout << " C  ";
    else if (i == highPos) std::cout << " H  ";
    else if (i == lowPos) std::cout << " L  ";
    else std::cout << " |  ";
    std::cout << '\n';
}
```

This part iterates through rows and prints H, O, C, and L characters to represent high, open, close, and low temperature values.

Main function:

```cpp
// Main function
int main() {
    std::string filePath = "weather_data_EU_1980-2019_temp_only.csv";

    // Get the country code from the user
    std::string location;
    std::cout << "Enter the country code (e.g., GB_temperature): ";
    std::cin >> location;

    auto candlestickData = computeCandlestickData(location, filePath);

    if (candlestickData.empty()) {
        std::cerr << "No data available for the specified country." << std::endl;
        return 1;
    }

    printVerticalCandlestickPlot(candlestickData);

    return 0;
}
```

This part handles user input, computes candlestick data, and visualizes the results.

Commands:

```
PS C:\Y2S1\object-oriented program\oop> g++ task2.cpp -o main
PS C:\Y2S1\object-oriented program\oop> ./main
```

Prompt:

```
Enter the country code (e.g., GB_temperature): GB_temperature
```

Output:

```
Legend:
  H - High temperature
  O - Open (average temperature of the previous year)
  C - Close (average temperature of the current year)
  L - Low temperature
-----------------------------
Year 2019:
  Open: 17.2185 | Close: 17.1663 | High: 36.039 | Low: -1.237
  40  H
  30  |
      |
  20  |
      O
      C
  10  |
      |
   0  |
      L
```

Task 3

Under Task3.cpp file

It computes Candlestick Data similarly to task 1 and 2.

```cpp
// Function to filter candlestick data by date range
std::vector<Candlestick> filterByDateRange(const std::vector<Candlestick>& data, int startYear, int endYear) {
    std::vector<Candlestick> filteredData;
    for (const auto& c : data) {
        if (c.year >= startYear && c.year <= endYear) {
            filteredData.push_back(c);
        }
    }
    return filteredData;
}
```

This part loop through the candlestick data and filters it to include only years within a specified range.

```cpp
// Function to filter candlestick data by temperature range
std::vector<Candlestick> filterByTemperatureRange(const std::vector<Candlestick>& data, double minTemp, double maxTemp) {
    std::vector<Candlestick> filteredData;
    for (const auto& c : data) {
        if (c.high <= maxTemp && c.low >= minTemp) {
            filteredData.push_back(c);
        }
    }
    return filteredData;
}
```

This part loop through the candlestick data and filters it where both high and low temperatures fall within a specified range.

Tabular Display

```cpp
// Function to display candlestick data in a tabular format
void displayCandlestickData(const std::vector<Candlestick>& data) {
    if (data.empty()) {
        std::cout << "No data to display.\n";
        return;
    }

    // Print table header
    std::cout << std::setw(8) << "Year" << std::setw(12) << "Open"
              << std::setw(12) << "Close" << std::setw(12) << "High"
              << std::setw(12) << "Low" << "\n";
    std::cout << std::string(56, '-') << "\n";

    // Print data rows
    for (const auto& c : data) {
        std::cout << std::setw(8) << c.year
                  << std::setw(12) << std::fixed << std::setprecision(2) << c.open
                  << std::setw(12) << std::fixed << std::setprecision(2) << c.close
                  << std::setw(12) << std::fixed << std::setprecision(2) << c.high
                  << std::setw(12) << std::fixed << std::setprecision(2) << c.low
                  << "\n";
    }
}
```

This part checks If the input vector is empty, it prints a message and exits. It uses std::setw to format the columns with consistent widths and prints the values for year, open, close, high, and low with fixed-point precision (2 decimal places).

```cpp
// Function to create a text-based graph
void plotTextGraph(const std::vector<Candlestick>& data, int plotHeight = 10) {
    if (data.empty()) {
        std::cout << "No data to plot.\n";
        return;
    }
```

This part handles the empty data. If empty, prints "No data to plot" and exits the function.

```cpp
    // Determine the global min and max temperatures for scaling
    double globalMin = std::numeric_limits<double>::max();
    double globalMax = std::numeric_limits<double>::lowest();

    for (const auto& c : data) {
        globalMin = std::min(globalMin, c.low);
        globalMax = std::max(globalMax, c.high);
    }
```

This part finds the overall lowest (globalMin) and highest (globalMax) temperatures across all candlestick data.

```
// Adjust globalMin and globalMax to the nearest multiple of 5
int roundedMin = static_cast<int>(std::floor(globalMin / 5.0)) * 5;
int roundedMax = static_cast<int>(std::ceil(globalMax / 5.0)) * 5;

// Ensure the tick interval is 5
int tickInterval = 5;
```

This part adjusts temperatures (y-axis) to a scale of 5 for better readability.

```
// Display temperature values above the graph
std::cout << "\nTemperature Values:\n";
for (const auto& c : data) {
    std::cout << "Year " << c.year << ": "
              << "Open: " << std::fixed << std::setprecision(1) << c.open << " | "
              << "Close: " << c.close << " | "
              << "High: " << c.high << " | "
              << "Low: " << c.low << "\n";
}
std::cout << "\n";
```

This part prints numerical temperature values (open, close, high, low) for each year before displaying the graph.

```
// Create a grid to store characters at each row and column
std::vector<std::vector<std::string>> grid(plotHeight + 1, std::vector<std::string>(data.size(), " "));
```

This part creates a 2D grid where rows represent temperature levels and columns represent years.

```
// Populate the grid with H, C, O, L based on actual values
for (size_t i = 0; i < data.size(); ++i) {
    const auto& c = data[i];

    // Calculate positions on the grid
    int highRow = static_cast<int>(std::round((c.high - roundedMin) / (roundedMax - roundedMin) * plotHeight));
    int closeRow = static_cast<int>(std::round((c.close - roundedMin) / (roundedMax - roundedMin) * plotHeight));
    int openRow = static_cast<int>(std::round((c.open - roundedMin) / (roundedMax - roundedMin) * plotHeight));
    int lowRow = static_cast<int>(std::round((c.low - roundedMin) / (roundedMax - roundedMin) * plotHeight));

    // Insert characters into the grid, ensuring no overlap
    if (grid[highRow][i] == " ") {
        grid[highRow][i] = "  H  ";
    }
    if (grid[closeRow][i] == " ") {
        grid[closeRow][i] = "  C  ";
    } else if (closeRow > 0 && c.close > c.high) {
        grid[closeRow - 1][i] = "  C  "; // Shift up
    }
    if (grid[openRow][i] == " ") {
        grid[openRow][i] = "  O  ";
    } else if (openRow > 0 && c.open > c.close) {
        grid[openRow - 1][i] = "  O  "; // Shift up
    }
    if (grid[lowRow][i] == " ") {
        grid[lowRow][i] = "  L  ";
    } else if (lowRow > 0 && c.low > c.open) {
        grid[lowRow - 1][i] = "  L  "; // Shift up
    }
}
```

This part maps high, close, open, and low temperatures to specific rows in the grid based on their scaled positions and each temperature value is scaled relative to roundedMin and roundedMax to fit within plotHeight.

```
// Generate the graph
for (int row = plotHeight; row >= 0; --row) {
    int currentValue = roundedMin + (row * tickInterval);

    std::cout << std::setw(6) << currentValue << " |";
    for (size_t col = 0; col < data.size(); ++col) {
        // Print centered characters in the grid
        std::cout << std::setw(6) << grid[row][col];
    }
    std::cout << "\n";

    // Add an empty row for spacing, except for the last row
    if (row > 0) {
        std::cout << std::setw(6) << " " << " |";
        for (size_t col = 0; col < data.size(); ++col) {
            std::cout << std::setw(6) << " ";
        }
        std::cout << "\n";
    }
}
```

This part prints the graph row by row from top to bottom. It prints the temperature label (currentValue) on the left and the corresponding markers (H, C, O, L) for each year or empty spaces if no marker exists. It also adds a blank row for spacing, except after the last row.

The main function is similar to task 1 and 2 except menu logic:

```
while (true) {
    // Display filter options
    std::cout << "\nFilter Options:\n";
    std::cout << "1. Filter by Date Range\n";
    std::cout << "2. Filter by Temperature Range\n";
    std::cout << "3. Reset Filters\n";
    std::cout << "4. Plot Data\n";
    std::cout << "5. Display Data\n";
    std::cout << "6. Exit\n";
    std::cout << "Enter your choice: ";

    int choice;
    std::cin >> choice;
```

```
    if (choice == 1) {
        int startYear, endYear;
        std::cout << "Enter start year: ";
        std::cin >> startYear;
        std::cout << "Enter end year: ";
        std::cin >> endYear;
        filteredData = filterByDateRange(filteredData, startYear, endYear);
        std::cout << "Data filtered by date range.\n";
```

Option 1: Prompts the user for the start and end years.

```
} else if (choice == 2) {
    double minTemp, maxTemp;
    std::cout << "Enter minimum temperature: ";
    std::cin >> minTemp;
    std::cout << "Enter maximum temperature: ";
    std::cin >> maxTemp;
    filteredData = filterByTemperatureRange(filteredData, minTemp, maxTemp);
    std::cout << "Data filtered by temperature range.\n";
```

Option 2: Prompts the user for the minimum and maximum temperatures.

```
} else if (choice == 3) {
    filteredData = candlesticks;  // Reset filters
    std::cout << "Filters reset.\n";
```

Option 3: Resets filteredData to the original candlesticks' dataset, removing all applied filters.

```
} else if (choice == 4) {
    // Check if the year range exceeds 10 years
if (!filteredData.empty()) {
    int startYear = filteredData.front().year;
    int endYear = filteredData.back().year;

    if (endYear - startYear > 10) {
        std::cout << "Cannot display: Year range exceeds 10 years.\n";
    } else {
        plotTextGraph(filteredData);
    }
} else {
    std::cout << "Cannot display: No data available for the current filters.\n";
}
```

Option 4: Checks if the filtered dataset's year range exceeds 10 years. It will not display output for more than 10 years as the terminal can't fit. If the range is valid, calls plotTextGraph to display the data as a graph.

```
} else if (choice == 5) {
    displayCandlestickData(filteredData); // Display data in aligned format
```

Option 5: Calls displayCandlestickData to display the filtered dataset as a table.

```
} else if (choice == 6) {
    std::cout << "Exiting.\n";
    break;
} else {
    std::cout << "Invalid choice. Please try again.\n";
}
```

Option 6: Prints a message and terminates the program. If the user enters an invalid option, prompts them to try again.

Commands:

```
PS C:\Y2S1\object-oriented program\oop> g++ task3.cpp -o main
PS C:\Y2S1\object-oriented program\oop> ./main
```

Prompt:

```
Enter the country code (e.g., GB_temperature): GB_temperature
```

Filter options:

```
Filter Options:
1. Filter by Date Range
2. Filter by Temperature Range
3. Reset Filters
4. Plot Data
5. Display Data
6. Exit
Enter your choice: 1
```

```
Enter start year: 1995
Enter end year: 1996
Data filtered by date range.

Filter Options:
1. Filter by Date Range
2. Filter by Temperature Range
3. Reset Filters
4. Plot Data
5. Display Data
6. Exit
Enter your choice: 4
```

Ouput:

```
Temperature Values:
Year 1995: Open: 16.7 | Close: 15.8 | High: 34.0 | Low: -0.9
Year 1996: Open: 15.8 | Close: 15.4 | High: 35.9 | Low: -0.5

    45 |
       |
    40 |   H     H
       |
    35 |
       |
    30 |
       |
    25 |
       |
    20 |   C     C
       |
    15 |   O     O
       |
    10 |
       |
     5 |
       |
     0 |   L     L
       |
    -5 |
         1995  1996
```

Task 4

Under task4.cpp file

```cpp
// Function to read and process the file for a specific country
std::map<int, double> readDataForCountry(const std::string& filePath, const std::string& country) {
    std::ifstream file(filePath);
    if (!file.is_open()) {
        std::cerr << "Error: Unable to open file: " << filePath << std::endl;
        return {};
    }

    std::string line;
    std::map<int, std::vector<double>> yearlyTemps;

    // Parse the header row to find the column for the specified country
    std::getline(file, line);
    std::stringstream headerStream(line);
    std::string columnName;
    int countryColumnIndex = -1;
    int currentIndex = 0;

    while (std::getline(headerStream, columnName, ',')) {
        if (columnName == country) {
            countryColumnIndex = currentIndex;
            break;
        }
        currentIndex++;
    }
```

This part opens the file and parse the Header Row. It reads the first line (header row) of the file to determine the column index for the specified country.

```cpp
// Process the data for the selected country
while (std::getline(file, line)) {
    std::stringstream lineStream(line);
    std::string date, tempStr;
    std::getline(lineStream, date, ',');

    int year = std::stoi(date.substr(0, 4));
    for (int i = 0; i <= countryColumnIndex; ++i) {
        std::getline(lineStream, tempStr, ',');
    }

    double temperature = safeStringToDouble(tempStr);
    if (!std::isnan(temperature)) {
        yearlyTemps[year].push_back(temperature);
    }
}
```

This part processes each row in the file to extract temperature data for the selected country.

```cpp
// Calculate the average temperature per year
std::map<int, double> avgTemps;
for (const auto& entry : yearlyTemps) {
    const auto& temps = entry.second;
    avgTemps[entry.first] = std::accumulate(temps.begin(), temps.end(), 0.0) / temps.size();
}
```

This part computes the average temperature for each year using the data in yearlyTemps.

```
    return avgTemps;
```

This part returns a map where the key is the year (int) and the value is the average temperature for that year (double).

```
// Function to compute polynomial regression coefficients
std::vector<double> fitPolynomial(const std::vector<int>& x, const std::vector<double>& y, int degree) {
    int n = x.size();
    int d = degree + 1;

    // Initialize matrices
    std::vector<std::vector<double>> X(d, std::vector<double>(d, 0));
    std::vector<double> Y(d, 0);

    // Compute X matr  f powers of x)
    for (int i = 0; i  (int)0  ) {
        for (int j = 0; j < d; ++j) {
            for (int k = 0; k < n; ++k) {
                X[i][j] += std::pow(x[k], i + j);
            }
        }
    }

    // Compute Y vector (sum of x^i * y)
    for (int i = 0; i < d; ++i) {
        for (int k = 0; k < n; ++k) {
            Y[i] += std::pow(x[k], i) * y[k];
        }
    }
```

```
    // Solve for coefficients using Gaussian elimination
    std::vector<double> coefficients(d, 0);
    for (int i = 0; i < d; ++i) {
        // Normalize the pivot row
        double pivot = X[i][i];
        for (int j = 0; j < d; ++j) {
            X[i][j] /= pivot;
        }
        Y[i] /= pivot;

        // Eliminate other rows
        for (int k = 0; k < d; ++k) {
            if (k != i) {
                double factor = X[k][i];
                for (int j = 0; j < d; ++j) {
                    X[k][j] -= factor * X[i][j];
                }
                Y[k] -= factor * Y[i];
            }
        }
    }

    // Extract coefficients
    for (int i = 0; i < d; ++i) {
        coefficients[i] = Y[i];
    }

    return coefficients;
```

This part fits a polynomial to the given data points and computes its coefficients where

x: Vector of years.

y: Vector of corresponding temperatures.

degree: Degree of the polynomial.

It constructs the X matrix and Y vector using sums of powers of x and products of x with y and solves the system of linear equations to find the polynomial coefficients.

```
// Function to predict temperature using polynomial regression
double predictPolynomial(const std::vector<double>& coefficients, int x) {
    double prediction = 0.0;
    for (size_t i = 0; i < coefficients.size(); ++i) {
        prediction += coefficients[i] * std::pow(x, i);
    }
    return prediction;
}
```

This part predicts the temperature for a given year using the polynomial regression model where
coefficients: Polynomial coefficients.

x: Year for which the prediction is made.

It computes the polynomial value for the given year using the formula:

$$y = a_0 + a_1 \cdot x + a_2 \cdot x^2 + \cdots$$

```
// Function to plot the predictions
void plotPredictions(const std::vector<std::pair<int, double>>& predictions) {
    if (predictions.empty()) {
        std::cout << "No predictions to plot.\n";
        return;
    }

    // Determine min and max temperature and round to nearest multiples of 0.05
    double minTemp = std::numeric_limits<double>::max();
    double maxTemp = std::numeric_limits<double>::lowest();
    for (const auto& p : predictions) {
        minTemp = std::min(minTemp, p.second);
        maxTemp = std::max(maxTemp, p.second);
    }
    double roundedMinTemp = std::floor(minTemp / 0.05) * 0.05;
    double roundedMaxTemp = std::ceil(maxTemp / 0.05) * 0.05;

    // Ensure a consistent tick interval of 0.05
    double tickInterval = 0.05;

    // Calculate the number of rows for the y-axis
    int numTicks = static_cast<int>((roundedMaxTemp - roundedMinTemp) / tickInterval) + 1;
```

This part creates a text-based plot of the predicted temperatures where

Determine Plot Range: Finds the minimum and maximum predicted temperatures and rounds the range to the nearest multiples of 0.05.

Grid Construction: Divides the temperature range into rows based on the tick interval (0.05).

```
// Create a grid for the plot
const int columnWidth = 8;
const int plotHeight = numTicks - 1;

for (int row = plotHeight; row >= 0; --row) {
    double currentValue = roundedMinTemp + row * tickInterval;

    // Print y-axis label
    std::cout << std::setw(6) << std::fixed << std::setprecision(2) << currentValue << " |";

    // Plot the points for each year
    for (const auto& p : predictions) {
        // Calculate the row for this prediction
        int predictedRow = static_cast<int>((p.second - roundedMinTemp) / tickInterval + 0.5);

        // Prepare column for alignment
        std::string column(columnWidth, ' ');
        if (row == predictedRow) {
            column[columnWidth / 2] = '*'; // Place the '*' in the middle
        }
        std::cout << column;
    }
    std::cout << "\n";
}

// Print the x-axis (year labels)
std::cout << "       "; // Offset for y-axis labels
for (const auto& p : predictions) {
    std::cout << std::setw(columnWidth) << p.first;
}
std::cout << "\n";
}
```

Populate the Plot: Places a * in the grid for each predicted temperature.

It will print the grid row by row and displays year labels aligned below the plot.

Main function:

The input and reading of data work the same as task 1.

```
std::vector<int> years;
std::vector<double> temperatures;
for (const auto& entry : avgTemps) {
    years.push_back(entry.first);
    temperatures.push_back(entry.second);
}
```

This part converts the avgTemps map into two separate vectors where
years: Contains all years with data.

temperatures: Contains the corresponding average temperatures for those years.

```
int degree = 2; // Degree of the polynomial
auto coefficients = fitPolynomial(years, temperatures, degree);
```

This part fits a degree-2 polynomial (quadratic regression) to the historical temperature data and calls the fitPolynomial function, which computes and returns the polynomial coefficients.

```
int startYear, endYear;
std::cout << "Enter start year for prediction: ";
std::cin >> startYear;
std::cout << "Enter end year for prediction: ";
std::cin >> endYear;
```

This part prompts the user to specify the range of years (startYear to endYear) for which temperature predictions are required.

```
// Check if the year range exceeds 10
if (endYear - startYear > 10) {
    std::cout << "Year range exceeds 10 years.\n";
    return 0;
}

if (startYear <= 2019) {
std::cout << "Cannot predict: Year range includes 2019 or earlier.\n";
return 0;
}
```

This part ensures the prediction range does not exceed 10 years for practical visualization and ensures the prediction range starts after 2019, as data before 2020 is historical and cannot be predicted.

```
// Perform predictions if range is valid
std::vector<std::pair<int, double>> predictions;
for (int year = startYear; year <= endYear; ++year) {
    double predictedTemp = predictPolynomial(coefficients, year);
    predictions.emplace_back(year, predictedTemp);
}
```

This part loops through each year in the specified range and predicts the temperature using the polynomial regression model. It stores the results as (year, temperature) pairs in the predictions vector.

```
std::cout << "\nPredicted Temperatures:\n";
for (const auto& p : predictions) {
    std::cout << "Year: " << p.first << ", Predicted Avg Temp: " << p.second << "°C\n";
}
```

This part prints the predicted temperatures for each year in the specified range in a clear and readable format.

```
std::cout << "\nPrediction Plot:\n";
plotPredictions(predictions);
```

This part calls the plotPredictions function to generate a text-based graph of the predicted temperatures.

Commands:

```
PS C:\Y2S1\object-oriented program\oop> g++ task4.cpp -o main
PS C:\Y2S1\object-oriented program\oop> ./main
```

Prompt:

```
Enter the country code (e.g., GB_temperature): GB_temperature
```

```
Enter start year for prediction: 2025
Enter end year for prediction: 2030
```

Output:

```
Predicted Temperatures:
Year: 2025, Predicted Avg Temp: 17.4512┬ C
Year: 2026, Predicted Avg Temp: 17.5134┬ C
Year: 2027, Predicted Avg Temp: 17.5765┬ C
Year: 2028, Predicted Avg Temp: 17.6403┬ C
Year: 2029, Predicted Avg Temp: 17.705┬ C
Year: 2030, Predicted Avg Temp: 17.7704┬ C

Prediction Plot:
 17.80 |
 17.75 |                                          *
 17.70 |                                    *
 17.65 |                              *
 17.60 |                        *
 17.55 |
 17.50 |              *
 17.45 |     *
           2025     2026     2027     2028     2029     2030
```